Пространственные структуры данных

Пространственные структуры данных

Проблема: O(n²) - каждый с каждым

Допустим, у вас есть 10 объектов. Чтобы проверить коллизии между ними, нужно сравнить каждый с каждым: 10 × 9 / 2 = 45 проверок. Терпимо.

А если объектов 500? Уже 124 750 проверок на каждый кадр. 1000 объектов - почти полмиллиона. При 60 FPS это полмиллиона проверок 60 раз в секунду. Игра превращается в слайдшоу.

Вот как выглядит наивный подход:

Python C#

```python
# O(n^2) - check every pair
def check_all_collisions(objects):
    for i in range(len(objects)):
        for j in range(i + 1, len(objects)):
            if collides(objects[i], objects[j]):
                resolve(objects[i], objects[j])
```

Bullet hell, системы частиц, толпа врагов на экране - везде, где объектов много, наивный O(n^2) не работает. Нужен способ проверять только тех, кто рядом.

Идея: разделяй ... и властвуй? Разделяй пространство!

Ключевое наблюдение: объект в левом верхнем углу карты **не может** столкнуться с объектом в правом нижнем. Зачем их сравнивать?

Идея простая - разделим мир на области. Объект сравниваем только с соседями по области. Вместо n^2 проверок получаем гораздо меньше.

Ок. Все структуры ниже реализуют эту идею, но по-разному.

Сетка (Uniform Grid)

Самый простой вариант - разбить мир на одинаковые ячейки фиксированного размера. Каждая ячейка хранит список объектов, которые в ней находятся. Чтобы проверить коллизии для объекта - смотрим его ячейку и 8 соседних (оранжевая рамка на анимации выше).

Внутри - двумерный массив списков (`List<GameObject>[,]` / `list[list[]]`). Три метода:

insert - вычисляем индекс ячейки целочисленным делением и кладём объект в список:

```
cx = int(obj.x / cell_size)
cy = int(obj.y / cell_size)
cells[cy][cx].append(obj)
```

query - собираем объекты из текущей ячейки и 8 соседних (3×3 окрестность). Не забываем проверку границ:

```
for dy in [-1, 0, 1]:
    for dx in [-1, 0, 1]:
        if in_bounds(cy + dy, cx + dx):
            result += cells[cy + dy][cx + dx]
```

clear - очищаем все списки в начале каждого кадра (объекты двигаются, старые позиции неактуальны).

Сложность:

- **insert** - O(1): вычислить индекс, добавить в список.
- **query** - O(k), где k - количество объектов в 9 ячейках.

Когда работает: объекты распределены более-менее равномерно, размер ячейки ≈ размер типичного объекта.

Когда ломается: большинство объектов сгрудились в одной области - одна ячейка раздувается, получаем тот же $O(n^2)$.

Размер ячейки - критический параметр. Слишком маленький - объект попадает в несколько ячеек. Слишком большой - слишком много соседей в каждой ячейке.

QuadTree

А что, если не хотим заранее выбирать размер ячейки? Или объекты распределены неравномерно - в городе густо, в поле пусто?

QuadTree (дерево квадрантов) решает это рекурсивным делением. Начинаем с одного большого прямоугольника - всего мира. Когда в нём набирается больше N объектов - делим на 4 части. Каждая часть, набрав N объектов, делится снова. И так далее.

Каждый узел хранит: `bounds` (прямоугольник), `objects` (список), `children` (4 дочерних узла или пусто), `depth` . Константы: `MAX_OBJECTS = 4` , `MAX_DEPTH = 6` .

subdivide - делим прямоугольник на 4 равные части.

insert - спускаемся по дереву, определяя квадрант по центру узла (`obj.x < mid_x` , `obj.y < mid_y`). Если узел - лист и объектов > MAX_OBJECTS и глубина < MAX_DEPTH: вызываем `subdivide()` , перераспределяем объекты по детям.

query(area) - рекурсивно: собираем `objects` текущего узла + спускаемся в дочерние узлы, чьи `bounds` пересекаются с `area` (проверка AABB).

Сложность:

- **insert** - O(log n): спускаемся по дереву до нужного листа.
- **query** - O(log n + k): находим нужные квадранты, собираем k объектов.

Когда работает: неравномерное распределение - QuadTree адаптируется, разбивая густые области мельче.

Когда ломается: все объекты в одной точке - дерево уходит на максимальную глубину, и каждый лист всё равно содержит все объекты.

Здесь объекты - точки (только x, y). В реальных QuadTree у объекта есть размеры (AABB, к примеру), и он может попасть сразу в несколько квадрантов. И с этим что-то нужно делать, иначе некорректное определение родительского квадранта может привести к тому, что иногда объект будет "пропадать" для разных частей игры.

Пересоздавать QuadTree каждый кадр? Обновлять позиции объектов каждый кадр? Что лучше?

Пространственное хеширование

А что, если мир бесконечный или его границы заранее неизвестны? Обычная сетка требует фиксированных размеров мира - нужен двумерный массив `cells[rows][cols]` . Мир 10 000 × 10 000 при ячейке 64 = массив 156 × 156 = ~24 000 ячеек. Большинство пустые. А если мир вообще бесконечный (Minecraft)?

Пространственное хеширование (spatial hashing) решает это: вместо массива "все возможные ячейки" храним только **занятые**. Координата ячейки `(cx, cy)` превращается в ключ хеш-таблицы. Нет объектов в ячейке - нет записи. Появился объект в координатах (9999, -3000) - создалась одна запись, а не массив на весь мир.

Принцип тот же, что и в обычном `dict` / `Dictionary` : вычислить ключ за O(1), найти по нему список за O(1). Вся магия - в хеш-функции. Хороший хеш **разбрасывает** соседние ячейки по разным корзинам, чтобы избежать коллизий:

```
hash(cx, cy) = (cx * 92837111) ^ (cy * 689287499)
bucket_index = hash % table_size
```

Числа выглядят страшно, но это просто большие простые числа - они минимизируют вероятность того, что две разные ячейки попадут в одну корзину. Если хеш-функция плохая (например, `cx + cy`), соседние ячейки будут сталкиваться, и вместо O(1) получим O(n).

Хеш-таблица растёт и сжимается по мере необходимости. Персонаж ушёл из области - ячейки очищаются, память освобождается. Пришёл в новую - создаются новые. Сетка же хранит всё с самого начала.

Пространственный хеш - это Grid, где `cells[cy][cx]` заменён на `hash_map[(cx, cy)]`. Тот же `insert`, тот же `query` с 3×3 окрестностью. Единственное отличие - как хранятся корзины. Если мир фиксирован и компактен - сетка проще (прямой доступ по индексу быстрее хеша). Если мир огромен или растёт - хеш экономит память.

Какую структуру выбрать?

	Сетка	QuadTree	Пр. хеш
Сложность реализации	Простая	Средняя	Простая
insert	O(1)	O(log n)	O(1)
query	O(k)	O(log n + k)	O(k)
Память	Фиксированная (все ячейки)	Адаптивная	По требованию
Лучший случай	Равномерное распределение	Неравномерное, кластеры	Большой разреженный мир
Худший случай	Все в одной ячейке	Все в одной точке	Плохая хеш-функция
Когда выбрать	Фиксированный уровень, объекты ≈ одного размера	Объекты группируются, разный масштаб	Бесконечный мир, редкое заполнение

Обратите внимание: все три структуры реализуют **один и тот же интерфейс** - `insert()` и `query()`. Игровой цикл не знает, что внутри:

Python C#

```python
spatial.clear()
for obj in objects:
    spatial.insert(obj)
```

```
4
5
6
7
8
for obj in objects:
    for other in spatial.query(obj):
        if other is not obj and collides(obj, other):
            resolve(obj, other)
```

Это тот же паттерн, что и в системах частиц: одинаковый код, разные параметры. Здесь - одинаковый игровой цикл, разная структура данных внутри `spatial`. Хотите сменить Grid на QuadTree? Меняете одну строчку - создание `spatial`. Остальной код не трогаете.

Этот принцип - **программирование к интерфейсу, а не к реализации** - одна из самых полезных идей в проектировании. Мы работаем с тем, *что требуется*, а не с тем, *как это реализуется*. Вы столкнётесь с ней ещё не раз: в событийной модели, ECS, паттерне Command.

За пределами игр

Пространственные структуры - не только про игры. Те же идеи появляются в алгоритмах, которые вы будете (надеюсь) изучать:

- **KD-дерево** - обобщение QuadTree на произвольное число измерений. Используется для поиска ближайших соседей (KNN) в машинном обучении - кластеризация, рекомендательные системы.
- **BSP (Binary Space Partitioning)** - рекурсивное деление пространства плоскостью. Использовалось в Doom и Quake для определения видимых поверхностей. То же BSP, о котором упоминается в коллизиях.
- **R-деревья** - пространственные индексы в базах данных и ГИС. Запрос "найди все рестораны в радиусе 500м" - по сути тот же `query()`.
- **BVH (Bounding Volume Hierarchy)** - ускоряющая структура для трассировки лучей (ray tracing). Та же идея - не проверять луч с каждым полигоном, а быстро отсечь далёкие.

Т.е. QuadTree - это не "игровая штука". Это фундаментальная идея разделения пространства, которая работает от игровых коллизий до поиска в многомерных данных.

Итого

Тема	Где почитать подробнее
Хитбоксы, AABB, проверка столкновений	Коллизии
Порядок обновления, фазы коллизий	Порядок обновления
Отсечение невидимых объектов	Камера и отсечение

Тема	Где почитать подробнее
Data-oriented design, пул объектов	Системы частиц
Компоненты и системы	ECS
Поиск путей на графе	Уровни и поиск путей
Визуализация структур при отладке	Отладка и инструменты
Освещение и рейкастинг	Простое освещение

Powered by Wiki.js